

University of Asia Pacific
Department of Computer Science and Engineering
REPORT: 07

Course Code: CSE 402
Course Title: Operating System Lab

Submitted by:

Name: Rajash Majumdar

Student ID:23101144

Semester : 4th Year 1st Semester

Section: C2

Submitted to:

Atia Rahman Orthi,
Lecturer
University of Asia Pacific

1. Problem Statement

Implement the Shortest Seek Time First (SSTF) Disk Scheduling Algorithm using Python for the following scenario:
•Request Queue: 3, 60, 10, 25, 30, 4
•Initial Head Position: 41

Calculate the total disk head movement and represent the disk head execution sequence. Explain how SSTF selects the nearest request and helps improve disk scheduling efficiency.

2. Objective

The main objective of this experiment is to understand and implement the SSTF disk scheduling algorithm using Python. The experiment aims to select the disk request closest to the current head position, calculate the movement between consecutive requests, and determine the total head movement required to serve all requests.

3. Introduction

Disk scheduling is an important operating system technique used to decide the order in which disk I/O requests are processed. When multiple requests are waiting, an efficient scheduling algorithm can reduce unnecessary disk head movement and improve performance.

Shortest Seek Time First (SSTF) selects the request that requires the smallest movement from the current head position. After serving that request, the algorithm searches for the nearest remaining request and repeats the process until all requests are completed.

4. Algorithm Description

The SSTF algorithm works through the following steps:
1.Set the initial disk head position to 41.
2.Calculate the distance between the current head position and every pending request.
3.Select the request with the minimum distance.
4.Add the distance to the total head movement.
5.Move the head to the selected request and remove it from the pending queue.
6.Repeat these steps until all requests have been served.

The distance between two positions is calculated using:

$$
\text{Movement}=|\text{Current Head}-\text{Next Request}|
$$

5. Source Code

The Python program uses a function named sstf() to select the nearest request, update the current head position, and calculate the total movement. The program finally prints the execution sequence and total head movement.

6. Result and Calculation

Initial Head Position: 41

Execution Sequence:
41 → 30 → 25 → 10 → 4 → 3 → 60

Head Movement	Distance
41 → 30	11
30 → 25	5
25 → 10	15
10 → 4	6
4 → 3	1
3 → 60	57
Total Movement	95

Therefore, the total disk head movement is **95** cylinders.

7. Discussion

In this experiment, the SSTF algorithm selected the nearest disk request and calculated the total head movement as 95 cylinders. Students may face difficulties in selecting the nearest request and calculating the distance between head positions. SSTF reduces disk movement, but requests far from the current head position may experience longer waiting times, known as starvation.

8. Conclusion

The SSTF disk scheduling algorithm was successfully implemented using Python. The execution sequence was 41 → 30 → 25 → 10 → 4 → 3 → 60, with a total head movement of 95 cylinders. This experiment helped us understand how SSTF selects the nearest request to improve disk scheduling efficiency.
Through this experiment, we learned how SSTF selects the closest pending request and calculates the distance travelled by the disk head. We also understood how an appropriate disk scheduling algorithm can reduce unnecessary movement and improve disk access efficiency. Although SSTF can provide efficient performance, it may cause starvation for requests that remain far from the current head position.

GitHub Repository link :
https://github.com/Rajash144/OS-CPU-Scheduling-Labs